

作业 1

张乐

2025 年 3 月 17 日

编程部分

1 基础任务

1.1 数据载入

为了验证数据载入文件的正确性，执行 python 文件 datasets.py：

```
length of train data: 2080
length of val data: 520
length of test data: 780
```

图 1: 数据载入部分的验证

训练集、验证集、测试集可以正常载入。根据输出结果可知，训练集的数据数为 2080、验证集的数据数为 520、测试集的数据数为 780。

1.2 模型构建

为了验证模型构建的正确性，执行 python 文件 networks.py：

```
model: Classifier(
  (mlp): Sequential(
    (0): Linear(in_features=2048, out_features=256, bias=True)
    (1): ReLU()
    (2): Linear(in_features=256, out_features=26, bias=True)
  )
)
y: tensor([[ 0.0822, -0.1673,  0.0394,  0.4639, -0.0655,  0.1800, -0.0551, -0.1967,
  0.2721, -0.2255, -0.1238, -0.3264, -0.0442,  0.2535,  0.1563, -0.0855,
  0.3233, -0.1944, -0.1797, -0.3977, -0.3753,  0.0831,  0.2573,  0.2700,
  0.0489, -0.0014]], grad_fn=<AddmmBackward0>)
```

图 2: 模型构建部分的验证

可以发现，程序正确构建了 2 层感知机并输出了单张图像的分类概率。

1.3 训练测试

为了验证训练测试函数的正确性，先执行 `python solve.py--mode train`：

```
strat training!  
[1/15]: train loss is 1.894335  
[1/15]: val loss is 0.677045, accuracy is 0.919231  
[2/15]: train loss is 0.285070  
[2/15]: val loss is 0.335389, accuracy is 0.923077  
[3/15]: train loss is 0.108865  
[3/15]: val loss is 0.238767, accuracy is 0.948077  
[4/15]: train loss is 0.059472  
[4/15]: val loss is 0.188147, accuracy is 0.946154  
[5/15]: train loss is 0.035689  
[5/15]: val loss is 0.186369, accuracy is 0.942308  
[6/15]: train loss is 0.023384  
[6/15]: val loss is 0.178627, accuracy is 0.948077  
[7/15]: train loss is 0.017075  
[7/15]: val loss is 0.155595, accuracy is 0.953846  
[8/15]: train loss is 0.014877  
[8/15]: val loss is 0.156447, accuracy is 0.959615  
[9/15]: train loss is 0.009609  
[9/15]: val loss is 0.137382, accuracy is 0.953846  
[10/15]: train loss is 0.007084  
[10/15]: val loss is 0.131710, accuracy is 0.957692  
[11/15]: train loss is 0.005623  
[11/15]: val loss is 0.127309, accuracy is 0.961538  
[12/15]: train loss is 0.004704  
[12/15]: val loss is 0.131810, accuracy is 0.955769  
[13/15]: train loss is 0.004047  
[13/15]: val loss is 0.134198, accuracy is 0.951923  
[14/15]: train loss is 0.003502  
[14/15]: val loss is 0.124455, accuracy is 0.959615  
[15/15]: train loss is 0.003088  
[15/15]: val loss is 0.120412, accuracy is 0.961538  
loss curve saved!
```

图 3: 训练过程

观察程序的训练过程可知，随着训练次数不断增多，训练集的损失迅速减小并最终趋近于 0；验证集的损失也有明显减小，但减小速度比训练集慢，最终验证集的平均准确率大于 96%。

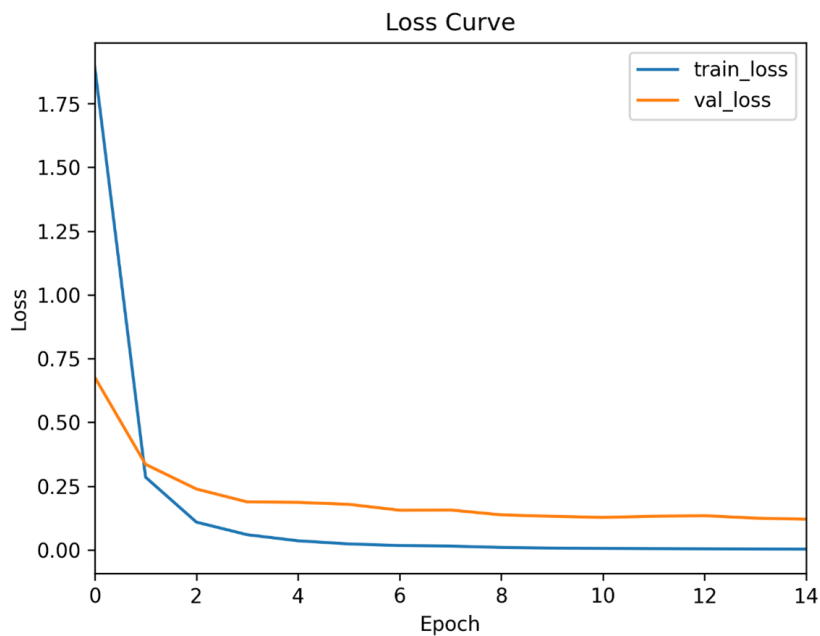


图 4: 损失函数曲线

损失函数曲线符合预期，是上述分析过程的可视化体现。

然后执行 `python solve.py--mode test`:

```
test accuracy is 0.957692
confusion matrix saved!
```

图 5: 测试过程

程序测试过程的平均准确率为 95.7692%，正确执行了测试过程。

同时，混淆矩阵符合预期：对于多类别分类的混淆矩阵，行的序号表示样本类别序号的真值，列的序号表示分类器预测的样本类别序号，归一化后混淆矩阵的对角线上数值趋近于 1，说明程序能对绝大多数数据进行准确识别和分类。通过观察混淆矩阵可知，此模型的性能较好。

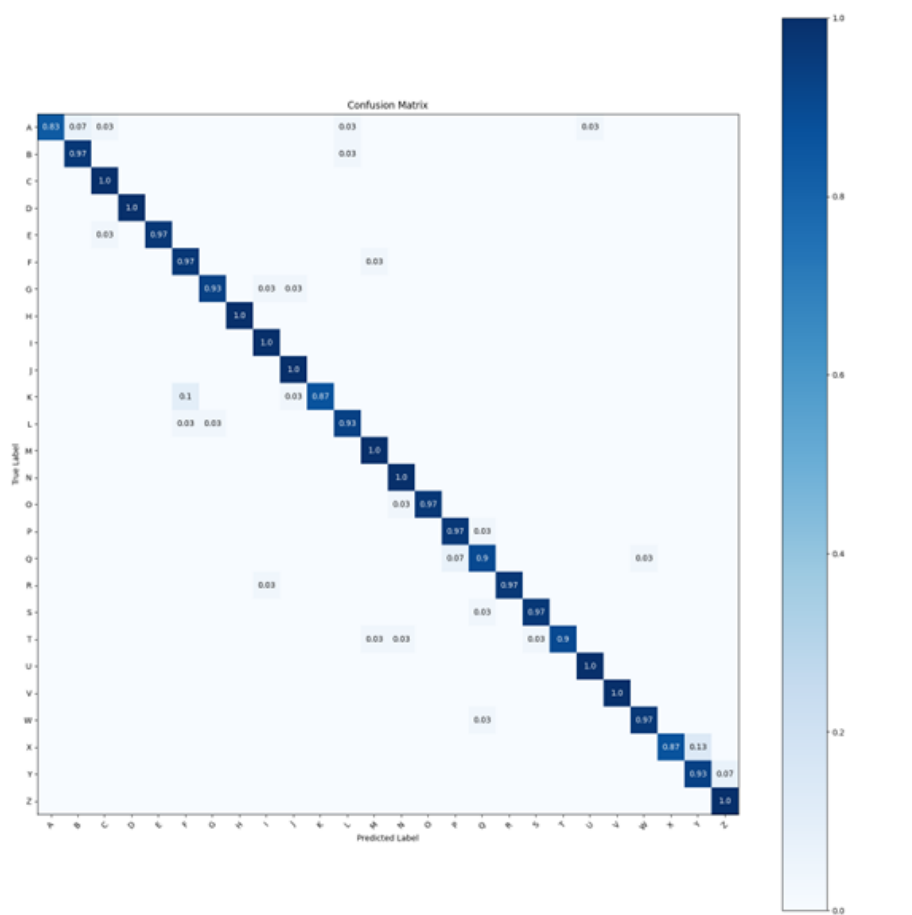


图 6: 混淆矩阵

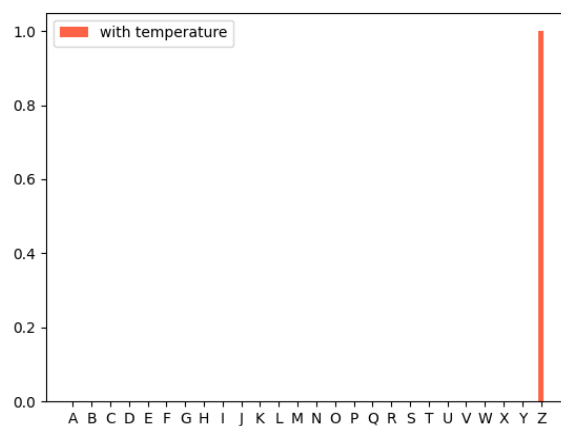
2 探索任务

选择了前 2 个探究任务。

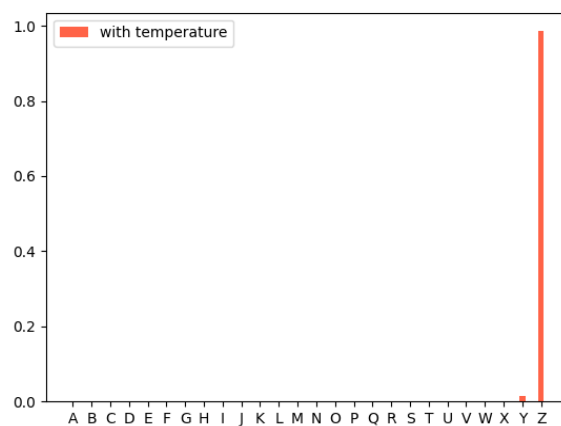
2.1 温度因子对 softmax 的影响

引入温度因子 T 后, softmax 函数为 $\text{softmax}(z_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$

将温度因子 T 引入 softmax, 令 $T=1e-2, 1, 1e2, 1e4, 1e6$, 记录不同 T 取值下任意一个样本 (实验中选取第一个样本) 经过 softmax 的分类输出概率, 分析 T 的改变对分类输出概率及测试平均准确率的影响及原因。

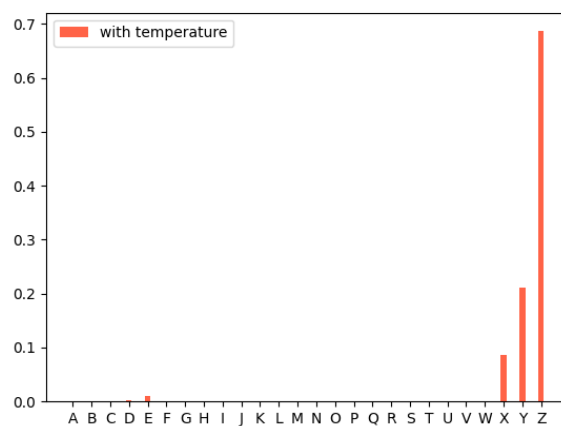
图 7: $T=1e-2$ 时分类输出概率

```
[15/15]: val loss is 0.593385, accuracy is 0.878846  
loss curve saved!  
softmax saved!
```

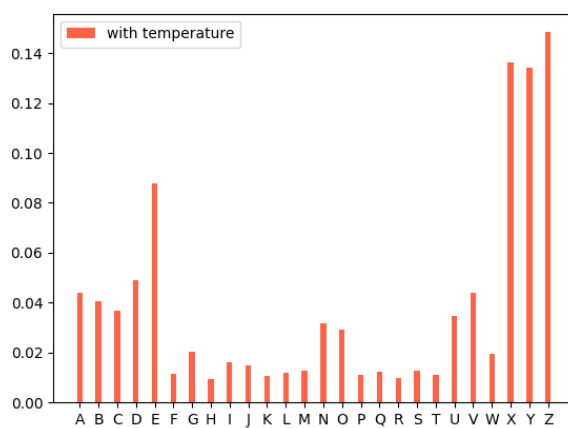
图 8: $T=1e-2$ 时测试平均准确率图 9: $T=1$ 时分类输出概率

```
[15/15]: val loss is 0.120412, accuracy is 0.961538  
loss curve saved!  
softmax saved!
```

图 10: $T=1$ 时测试平均准确率

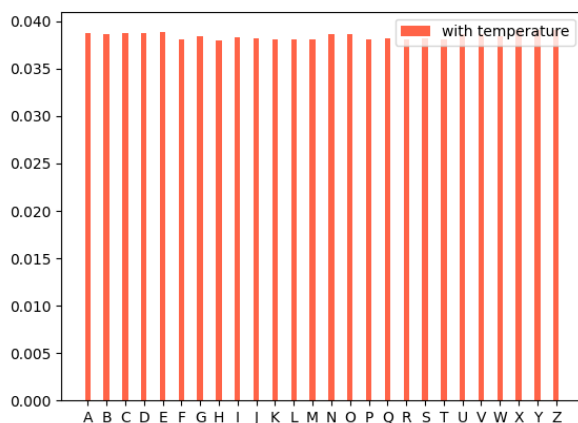
图 11: $T=1e2$ 时分类输出概率

```
[15/15]: val loss is 0.292659, accuracy is 0.934615  
loss curve saved!  
softmax saved!
```

图 12: $T=1e2$ 时测试平均准确率图 13: $T=1e4$ 时分类输出概率

```
[15/15]: val loss is 2.573037, accuracy is 0.576923  
loss curve saved!  
softmax saved!
```

图 14: $T=1e4$ 时测试平均准确率

图 15: $T=1e6$ 时分类输出概率

```
[15/15]: val loss is 3.246497, accuracy is 0.251923  
loss curve saved!  
softmax saved!
```

图 16: $T=1e6$ 时测试平均准确率

当 $T=1$ 时，退化为普通的 softmax 函数。

当 T 较高时 ($T>1$) 时，softmax 的输出分布更平均，模型对各类别的概率差异小。其原因是 $T>1$ 时 softmax 函数中指数部分较小，导致所有类别的概率值趋于接近。

当 T 较小时 ($T<1$) 时，softmax 的输出分布更集中于个别类，模型对各类别的概率差异大。其原因是 $T<1$ 时 softmax 函数中指数部分较大，导致最大概率值会更大，其他概率值会更小。

当 T 接近 1 时测试的平均准确率较高，当 T 远大于或远小于 1 时，测试的平均准确率较低。

2.2 自定义线性层与激活函数

重新训练并保存损失函数曲线、测试过程的混淆矩阵以及测试过程的平均准确率，与基础任务中的实现进行对比：

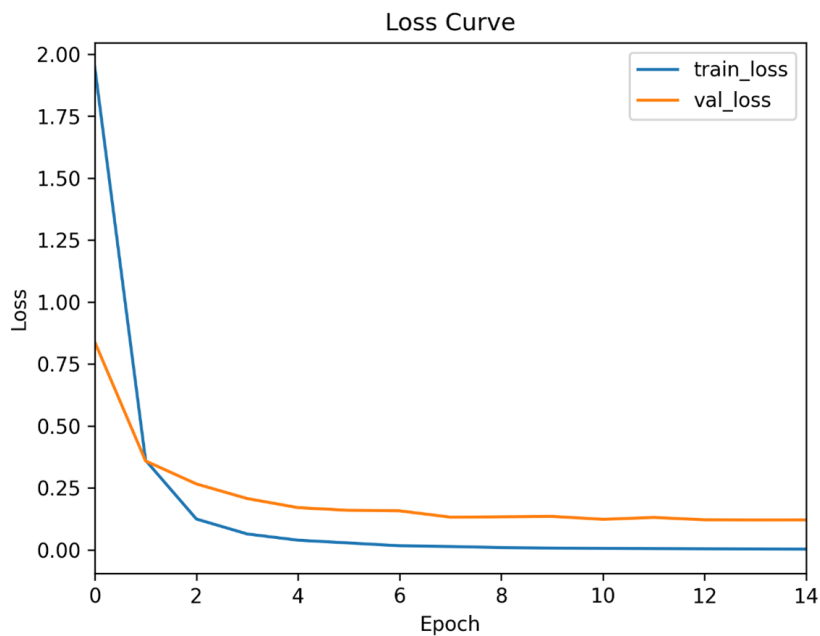


图 17: 新的损失函数曲线

由图可知，新的损失函数曲线整体趋势与原损失还是曲线一致，仅有细微区别。

```
test accuracy is 0.953846
confusion matrix saved!
```

图 18: 新的测试过程

而新的测试过程输出结果表明，自定义线性层与激活函数后，程序测试过程的平均准确率为 95.3846%，与原平均准确率 95.7692% 基本一致。

新的混淆矩阵形态也与基础模型的混淆矩阵基本一致，仍符合对角线上数值趋近于 1 的特点。但测试过程的准确率略低于基础模型，说明自定义线性层与激活函数后模型性能有所下降。

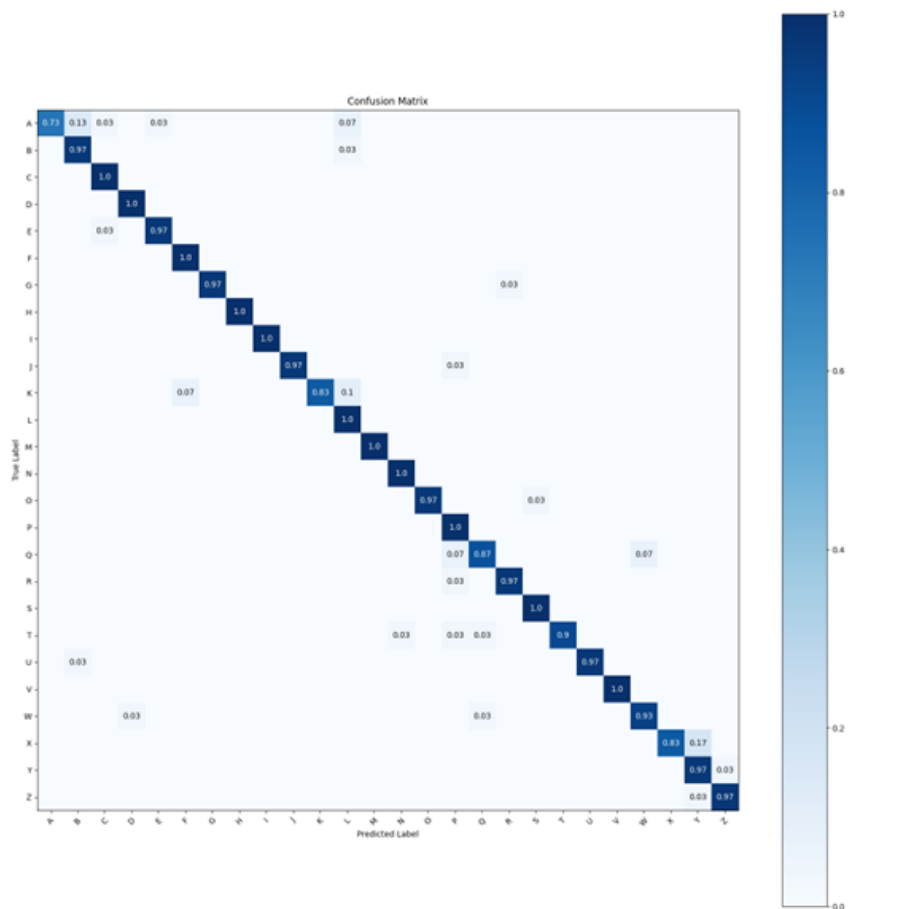


图 19: 新的混淆矩阵

综上所述，无论是损失函数曲线、测试过程的平均准确率，还是测试过程的混淆矩阵，自定义模型与基础模型的实验结果均相近，说明成功完成了线性层与激活函数的自定义实现。